

GeoVision

Final Reviewed Architecture & Implementation Plan

Smart India Hackathon 2026 | Problem Statement 1705
Theme: Robotics and Drones | Category: Software | Team: Recursion

This document reconciles two independently-generated technical proposals into one final, locked architecture. It supersedes prior drafts.

0. Review — Reconciling Two Conflicting Proposals

Two separate research passes were run on this project and produced genuinely different architectures. This section documents the conflict and the final decision, so the reasoning is on record.

0.1 Proposal A — Specialist Model Per Feature

Three separate, unrelated model families: **Attention U-Net++** for buildings, **D-LinkNet** (a different encoder-decoder family) for roads, and **Detectron2 / Mask R-CNN** (an instance-segmentation, box-based framework) for water bodies, plus a separate rooftop classifier.

0.2 Proposal B — Shared Multi-Task Network

One shared encoder-decoder backbone with a single multi-class segmentation head (background / building / road / water) and a secondary classification head running on cropped building regions for roof type.

0.3 Final Decision

LOCKED DECISION:

Build ONE shared-encoder, multi-head network (Proposal B's structure) — not three separate model families. Fold Proposal A's genuinely useful architectural insights into the heads of that shared network, rather than replacing the network with separate frameworks.

Reasoning:

- **Proposal A repeats Team Vaayu's own documented mistake.** Their loss on this exact PS came partly from stitching together unrelated model families (FPN + Detectron2) — a fragile, hard-to-maintain patchwork. Adding a third framework (Detectron2/Mask R-CNN) on top of two segmentation model families reproduces that exact risk, not a fix for it.
- **Engineering surface area matters under hackathon time pressure.** Three architectures means three training pipelines, three sets of hyperparameters, three checkpoints, three inference paths, and three independent failure modes to debug before finals. One shared network is dramatically lower-risk to actually finish and demo live.
- **The narrative is stronger and more defensible.** "One shared backbone, task-specific heads" is a clean, explainable architecture judges can follow in 30 seconds. "Three different frameworks bolted together" invites exactly the kind of Q&A scrutiny that sank the more complex reference submission.
- **Proposal A's best ideas aren't discarded — they're relocated.** D-LinkNet's real insight (dilated convolutions give a large receptive field for tile-spanning linear structures) becomes a dilated-conv module *inside* the road head. Attention U-Net++'s attention gates become part of the building head's skip connections. Detectron2's water performance in Team Vaayu's own tests is acknowledged as a real empirical finding — but addressed via loss design (class-balanced sampling + Tversky loss) inside the shared network's water head instead of a separate instance-segmentation framework.
- **Two ideas from Proposal A are kept as-is** because they don't conflict with the architecture and materially strengthen the project: realistic per-class accuracy targets (Section 4), and GI Cloud (MeghRaj) as the deployment target for data-sovereignty reasons (Section 6).

1. Problem Statement — Summary

PS 1705 — Development and Optimization of an AI model for Feature Identification/Extraction from drone orthophotos, under the **SVAMITVA Scheme** (Ministry of Panchayati Raj). SVAMITVA maps rural "Abadi" (inhabited) land using drone + CORS + GIS technology to issue legal property records; as of 2026 it covers roughly 3.3 lakh villages.

The PS requires an AI system that:

- Extracts **building footprints** from 50cm drone orthophotos, and classifies each rooftop as **RCC, Tiled, Tin, or Other** (feeding solar-potential and property-tax calculations)
- Extracts **road features**
- Extracts **waterbodies**
- Hits a **95% accuracy target**
- Is **optimized for efficient processing and deployment** as a cloud-based solution
- Is ready to **integrate into MoPR's existing systems**

Training basis: SVAMITVA drone-labeled datasets for 10 villages (small — this drives most of the data-strategy decisions in Section 5).

2. Final Model Architecture

One shared encoder, multiple task-specific decoder heads. A single pretrained backbone (ResNet34 / MobileNetV3, ImageNet-pretrained via *timm*) extracts general visual features once per tile; each head then specializes on top of those shared features.

Head	Design	Fix borrowed from Proposal A
Building head	U-Net-style decoder with attention-gated skip connections; boundary-aware term in the loss	Attention U-Net++'s attention gates — sharpens boundaries without a separate model
Road head	Same decoder family, with a dilated-convolution (ASPP-style) bottleneck module added; connectivity-aware loss term	D-LinkNet's core insight (large receptive field for tile-spanning linear structures) — without a second framework
Water head	Same decoder family; class-balanced sampling + Tversky loss (penalizes false negatives more); heavy color/texture augmentation	Addresses the algae-vs-vegetation confusion and data scarcity Team Vaayu documented, via loss design not a new framework
Roof-type head	Runs only on cropped building regions from the building head's output; lightweight EfficientNet-B0/ResNet18 classifier (transfer learning) -> RCC / Tiled / Tin / Other	Two-stage crop-and-classify — both proposals agreed on this part

All four heads sit on the same encoder and are trained jointly in one training run, producing one checkpoint, one ONNX export, and one inference path — not three separate models to serve and keep in sync.

2.1 Architecture Diagram (for hand-drawing / Eraser.io)

INPUT TILE (georeferenced, e.g. 512x512 RGB) → SHARED ENCODER (ResNet34/MobileNetV3, ImageNet-pretrained) → splits into four parallel branches: [Building Decoder + Attention Gates] · [Road Decoder + Dilated-Conv Bottleneck] · [Water Decoder + Class-Balanced Loss] → each outputs a segmentation mask for its class. The Building Mask branch additionally feeds: crop each detected building →

Roof-Type Classifier (EfficientNet-B0) → RCC / Tiled / Tin / Other label per building.

3. End-to-End System Architecture

Layer	What happens
1. Ingestion	User uploads ECW/GeoTIFF orthophoto via web app -> FastAPI receives it -> job pushed to Celery + Redis async queue (large files can't block an HTTP request)
2. Pre-processing	rasterio performs windowed reads/writes, converting to Cloud-Optimized GeoTIFF and tiling into overlapping 512x512 patches (10-15% overlap) -- CRS/affine-transform preserved per tile throughout, avoiding the georeferencing-loss bug seen in prior attempts at this PS
3. AI inference	Each tile passes through the shared-encoder multi-head network (Section 2) via ONNX Runtime; building crops are additionally routed through the roof-type classifier
4. Human verification	Output overlays are shown to a reviewing officer in the web map UI, who can edit/correct polygons before certification -- keeps the system honest about not claiming full autonomy, and matches how such tools are actually adopted operationally
5. Post-processing	Feather-blended stitching of overlapping tiles removes boundary seams; masks are polygonized (rasterio/shapely), simplified, and tagged with attributes (class, roof type, area)
6. Output & integration	Stored in PostGIS; exported as SHP / GeoJSON / KML -- formats already used by QGIS and MoPR's existing GIS workflows; raster layers served as COG tiles to the map viewer

4. Accuracy Strategy — Reporting Honestly

The PS states a "95% accuracy" target. On an imbalanced aerial scene (background pixels dominate every tile), overall pixel accuracy and per-class IoU are very different numbers -- and the difference is exactly what a technically literate judge will probe.

Be precise about which metric hits 95%:

Overall pixel accuracy (background-dominated) is realistically achievable near/above 95% -- comparable public benchmarks (WHU Buildings) report 97%+ pixel accuracy. **Per-class IoU is a harder, more honest metric** and should be reported separately with realistic targets: buildings ~85-90% IoU, roads ~65-75% IoU, water ~75-80% IoU. Claiming 95% IoU across all classes -- especially water -- would be an obvious red flag to anyone who has benchmarked these datasets before.

Methodology to make this defensible:

- Train/val/test split with no tile-level leakage between tiles from the same source image
- Report per-class IoU and F1 (building / road / water), not just an aggregate accuracy number
- Weighted cross-entropy + Dice loss for the segmentation heads; Tversky loss specifically for the water head
- Confusion matrix + a "worst-performing tiles" visualization, so failure modes are visible, not hidden
- Early stopping, LR scheduling, checkpointing on best validation IoU (not best loss)

5. Data Strategy

10 labeled villages is not enough to train any of the above from scratch. The fix is transfer learning from public, domain-matched benchmarks, then fine-tuning on real SVAMITVA data.

Feature	Pretraining dataset(s)	Why this dataset
Buildings	WHU Building Dataset, Inria Aerial	~0.3-0.45m resolution -- close match to SVAMITVA's 50cm drone imagery
Roads	Massachusetts Roads, DeepGlobe Road Extraction	Same domain (aerial, not 30m-resolution satellite)
Water	DeepGlobe Land Cover / public water-segmentation sets	Directly targets the scarce-class problem

Do first, before building anything else:

Pull the sample_data folder from the previously-referenced open-sourced repo (real SVAMITVA sample tiles + shapefiles) and run a quick stock baseline -- shared encoder, ImageNet-pretrained, no fancy transfer learning yet -- to see the real starting IoU on real data. That number determines whether the WHU/Massachusetts pretraining step is a nice-to-have or the thing standing between the team and a working demo.

Explicitly avoided: general-purpose satellite foundation models (e.g. Prithvi) built for ~30m Sentinel/HLS imagery -- wrong resolution domain for 50cm drone orthophotos. Raw zero-shot SAM/LangSAM for buildings is also avoided -- documented to over-segment and grab irrelevant objects on dense building clusters; a LoRA-fine-tuned SAM variant remains a valid benchmark experiment, not the default path.

6. Final Technology Stack

Layer	Technology
Model training	PyTorch + segmentation_models_pytorch (encoder-decoder with pretrained encoders built in), timm (backbones), Albumentations (geo-safe augmentation: rotation/flip/color jitter, no crops that break alignment)
Geo I/O	rasterio (windowed reads/writes, CRS preserved end-to-end) + GDAL for format conversion
Inference serving	ONNX Runtime -- exported, quantized (dynamic INT8) model; accuracy reported before/after quantization
Backend	FastAPI + Celery/Redis (async tile-inference job queue) + PostGIS
Storage / deployment target	GI Cloud (MeghRaj) / NIC-empanelled infrastructure -- land-record data has sovereignty implications a generic AWS/GCP deployment does not address; this is a genuine differentiator vs. both reference teams
Frontend	Next.js + Leaflet/Mapbox GL, with an editable vector layer so a reviewing officer can correct AI output before certification (Section 3, Layer 4)
Output formats	SHP, GeoJSON, KML -- matches existing QGIS/GIS workflows already in use

7. Explicitly Out of Scope

No blockchain, no AR/Unity visualization, no LLM chatbot, no multi-cloud deployment scaffolding. This is a model + working prototype, not a platform. Both add-ons were present in a prior reference team's submission and did not correlate with winning; effort here goes into the three segmentation heads, the roof classifier, and a genuinely working, geo-accurate, end-to-end pipeline instead.